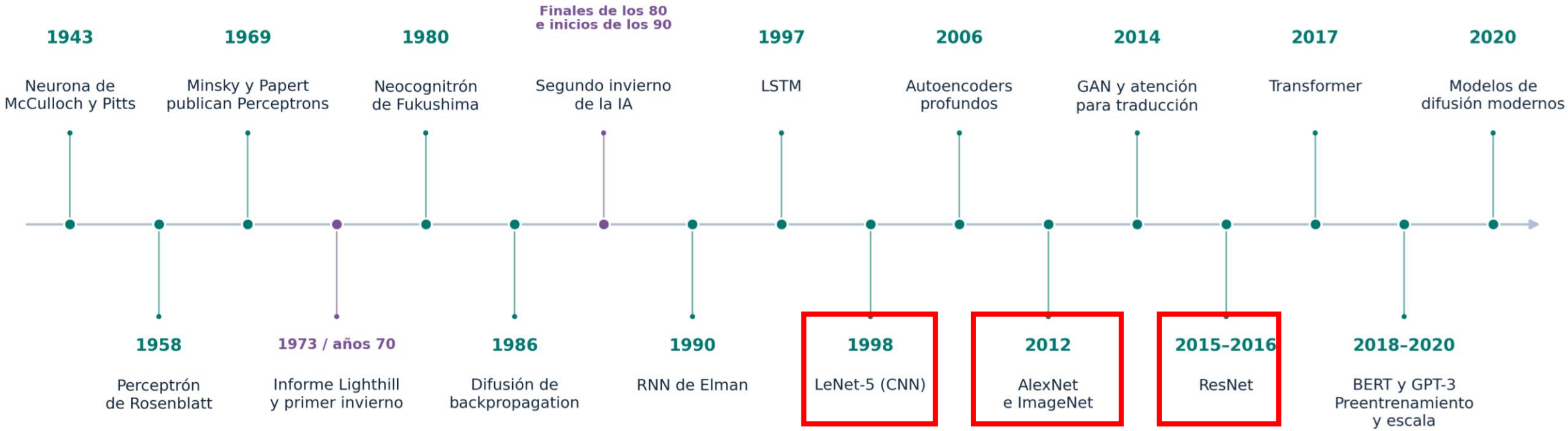
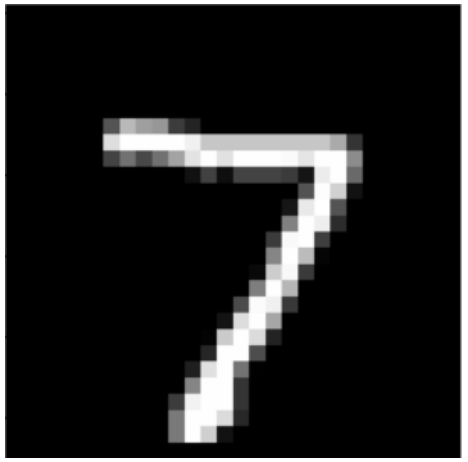
Redes Neuronales Convolucionales (CNN)

Joan Raygoza


1943
Neurona de McCulloch y Pitts
1958
Perceptrón de Rosenblatt
1969
Minsky y Papert publican Perceptrons
1973 / años 70
Informe Lighthill y primer invierno
1980
Neocognitrón de Fukushima
1986
Difusión de backpropagation
Finales de los 80 e inicios de los 90
Segundo invierno de la IA
1990
RNN de Elman
1997
LSTM
1998
LeNet-5 (CNN)
2006
Autoencoders profundos
2012
AlexNet e ImageNet
2014
GAN y atención para traducción
2015–2016
ResNet
2017
Transformer
2018–2020
BERT y GPT-3 Preentrenamiento y escala
2020
Modelos de difusión modernos
Hito técnico
Contexto histórico de la IA
Orden cronológico con espaciado uniforme

MLP y CNN

- Una imagen de 28 × 28 en gris contiene 784 valores. Una primera capa densa de 128 unidades tiene $784 \cdot 128 + 128 = 100\,480$ parámetros. Si la entrada es RGB, cada píxel aporta tres valores y la cantidad de pesos aumenta.
- Comúnmente se suelen usar imágenes RGB de 224 × 224, hay 150 528 pixeles/entradas. Conectar todas ellas a 256 unidades requiere **38 535 424 parámetros** en una sola capa.



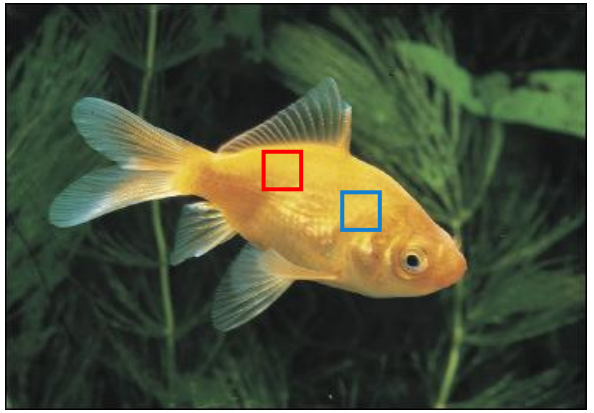
Ejemplo de Digits MNIST



Ejemplo de Imagenet

MLP y CNN

- En una imagen, píxeles vecinos suelen formar trazos, bordes y texturas. Una conexión densa permite relacionar cualquier píxel con cada unidad, **pero asigna un parámetro independiente a cada relación**. La arquitectura convolucional **incorpora una preferencia por relaciones locales y por reutilizar el mismo patrón**.
- Una MLP tiene capacidad para resolver tareas visuales, y es una referencia válida para conjuntos pequeños. La ventaja que buscamos en la CNN es una **estructura adecuada para imágenes, que puede facilitar el aprendizaje y reducir parámetros**.
- Tener **pocos parámetros no equivale automáticamente a necesitar pocas operaciones** ni a un menor tiempo de entrenamiento.

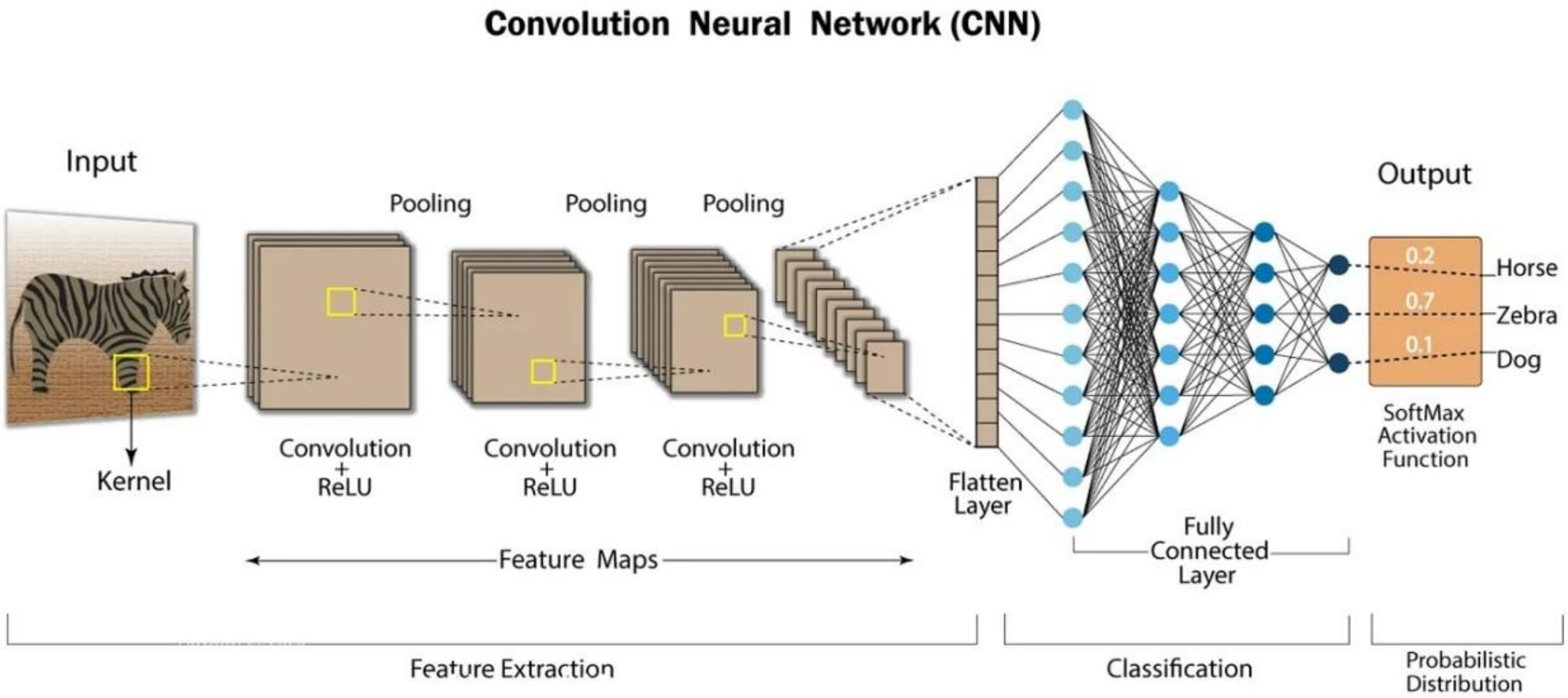


Ejemplo de Imagenet

LeNet-5

- En los noventa, una aplicación muy concreta del reconocimiento de imágenes era leer números escritos a mano en documentos.
- En 1998, Yann LeCun y sus colaboradores describieron **LeNet-5**, una red que aprendía de ejemplos qué trazos eran útiles para distinguir los dígitos.
- Net-5 tenía **5 194 conexiones**, pero solo **1 060 parámetros independientes**. Al aplicar los mismos filtros en distintas posiciones, muchas conexiones reutilizaban los mismos pesos. Así obtuvo la mayor exactitud de las cinco redes con el menor número de parámetros.

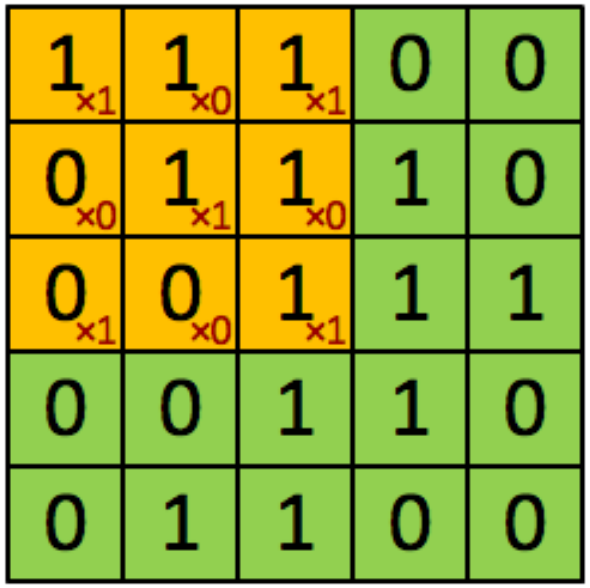
Modelo	Exactitud en prueba	Parámetros entrenables independientes
Net-1	80.0 %	2 570
Net-2	87.0 %	3 214
Net-3	88.5 %	1 226
Net-4	94.0 %	1 132
Net-5	**98.4 %**	**1 060**



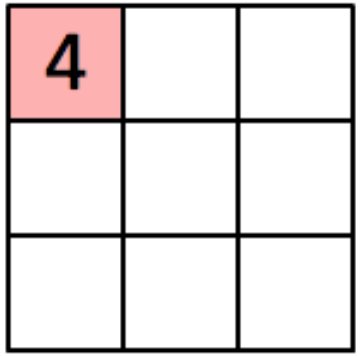
Arquitectura de una CNN

Localidad y pesos compartidos

- Una unidad de una **capa convolucional** mira una región de la entrada. Con un kernel de 3 × 3 sobre una imagen de un canal, su respuesta usa nueve valores vecinos. Al mover la ventana una columna, la siguiente unidad usa otra región. Los valores de los pesos se conservan.
- Esto permite distinguir **una unidad de salida**, asociada a una posición, de **un filtro aprendido**, reutilizado en muchas posiciones. La capa produce muchas activaciones, aunque el conjunto de parámetros sea pequeño.

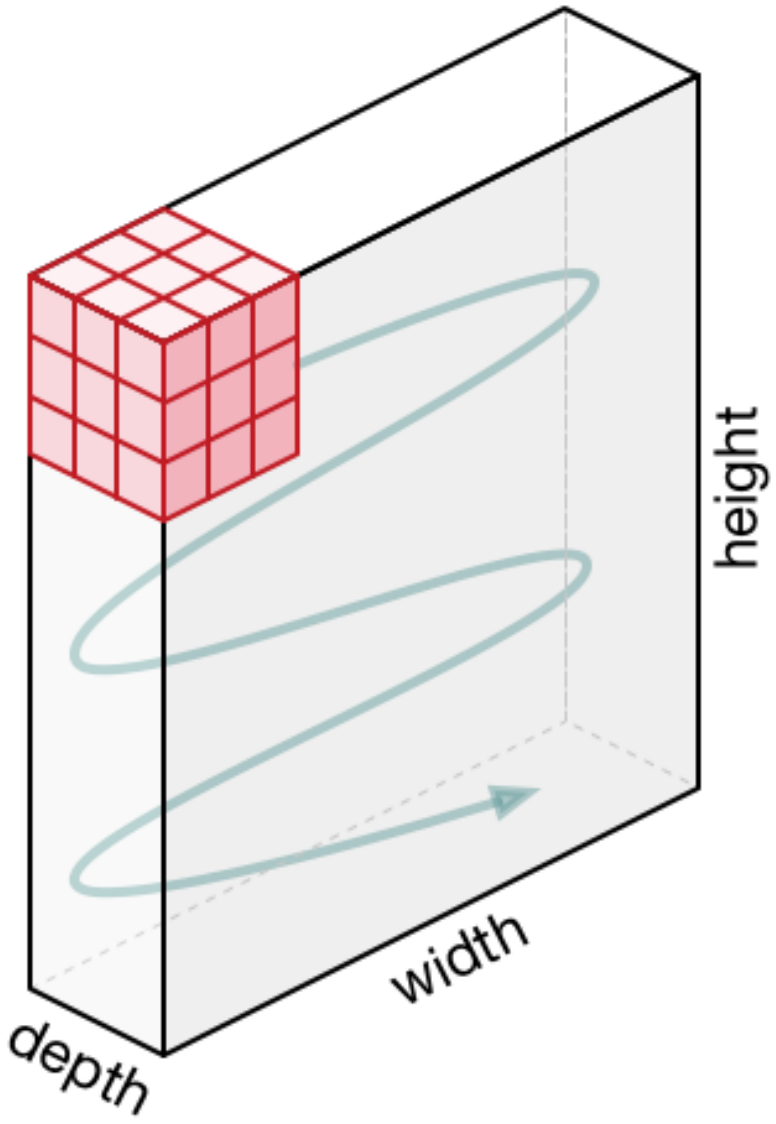


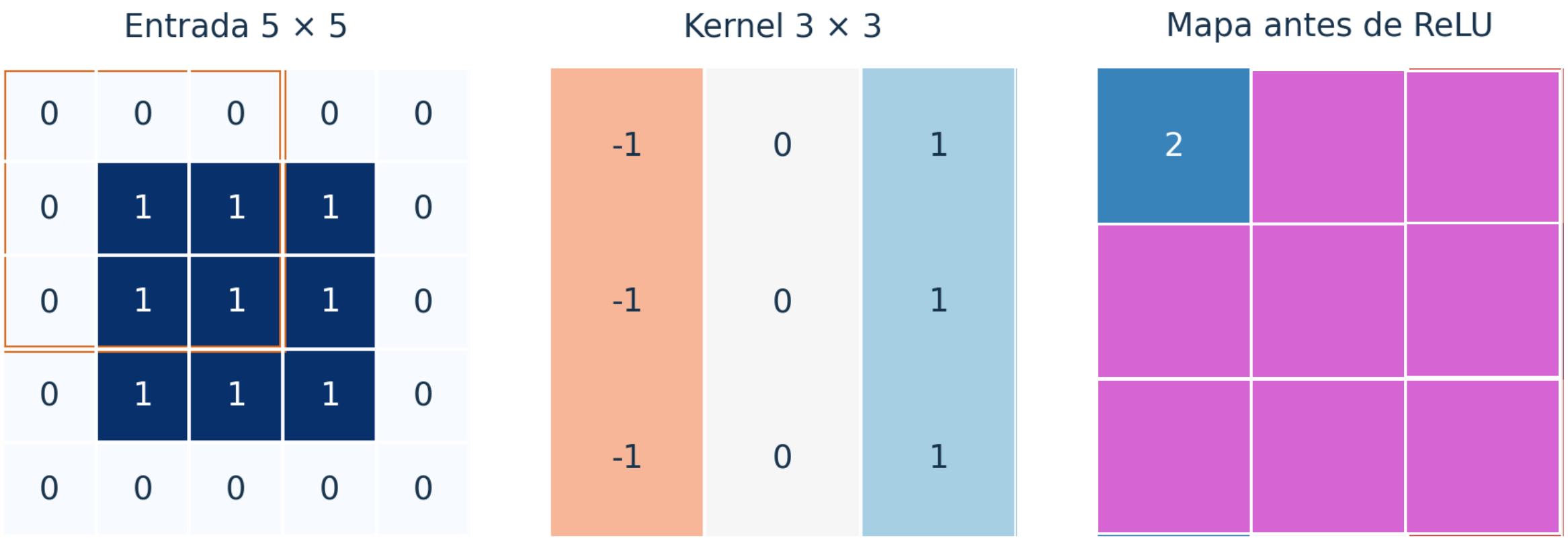
Después aplicar función de activación



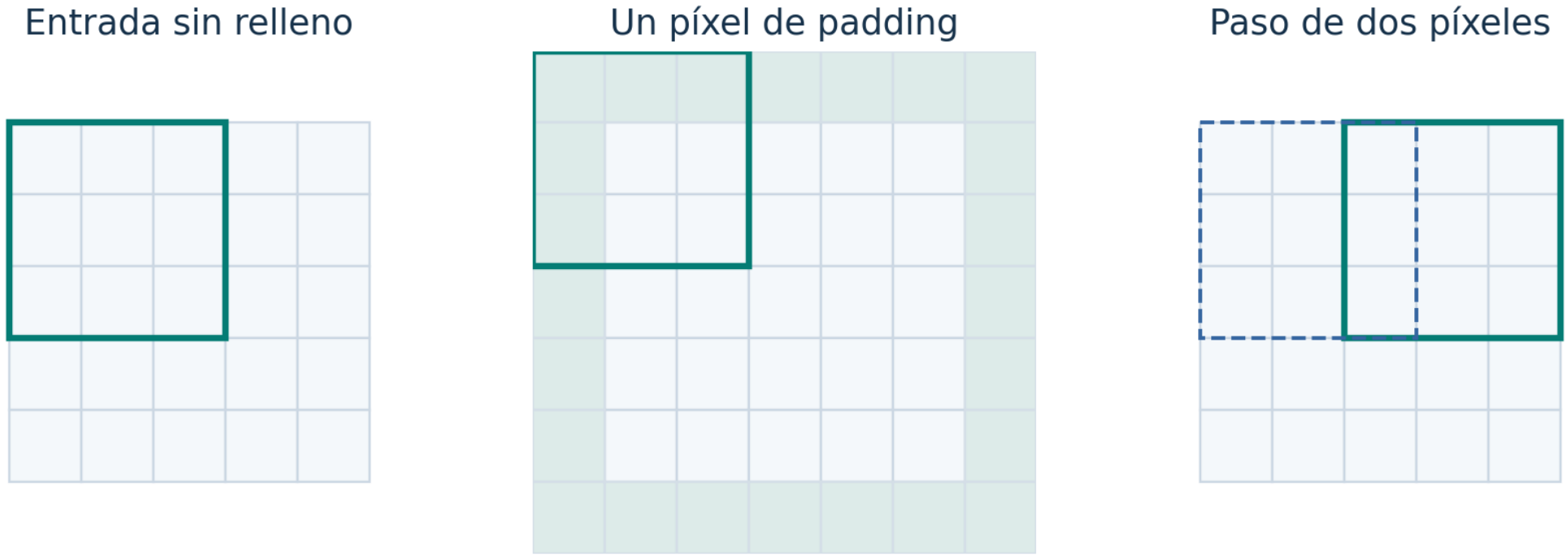
Localidad y pesos compartidos

- El filtro se desplaza hacia la derecha con un paso fijo hasta recorrer todo el ancho de la imagen.
- Después baja ese mismo número de píxeles, vuelve al extremo izquierdo y repite el recorrido hasta cubrir toda la imagen.





Ejemplo de convolución



Stride padding y tamaño de salida

- El **stride** es el número de posiciones que avanza la ventana por paso. Un stride de 1 produce respuestas en posiciones contiguas. Con stride 2 se omiten posiciones intermedias y normalmente se reduce la resolución del mapa.
- El **padding** agrega valores alrededor de la entrada. Con padding de ceros, la ventana puede operar cerca de los bordes sin salir del tensor. El relleno modifica las respuestas periféricas porque introduce valores que no provienen de la imagen.

Stride padding y tamaño de salida

- Para un kernel cuadrado de tamaño k, padding simétrico p, stride s y dilatación/separación d, el alto de salida es

$$H_{\text{out}} = \text{piso}\left(\frac{H + 2p - d(k-1) - 1}{s}\right) + 1.$$

- El ancho sigue la misma regla. La función piso redondea hacia abajo y cuenta solo ventanas completas que caben en la entrada rellenada.

Entrada y configuración	Salida espacial
5 × 5, kernel 3, padding 0, stride 1	3 × 3
5 × 5, kernel 3, padding 1, stride 1	5 × 5
5 × 5, kernel 3, padding 1, stride 2	3 × 3
28 × 28, kernel 3, padding 1, stride 1	28 × 28

Tamaño de salida de una convolución

Cuatro configuraciones con un kernel de 3 × 3

A Entrada 5 × 5

Kernel 3 × 3 · Padding 0 · Stride 1



B Entrada 5 × 5

Kernel 3 × 3 · Padding 1 · Stride 1



C Entrada 5 × 5

Kernel 3 × 3 · Padding 1 · Stride 2



D Entrada 28 × 28

Kernel 3 × 3 · Padding 1 · Stride 1



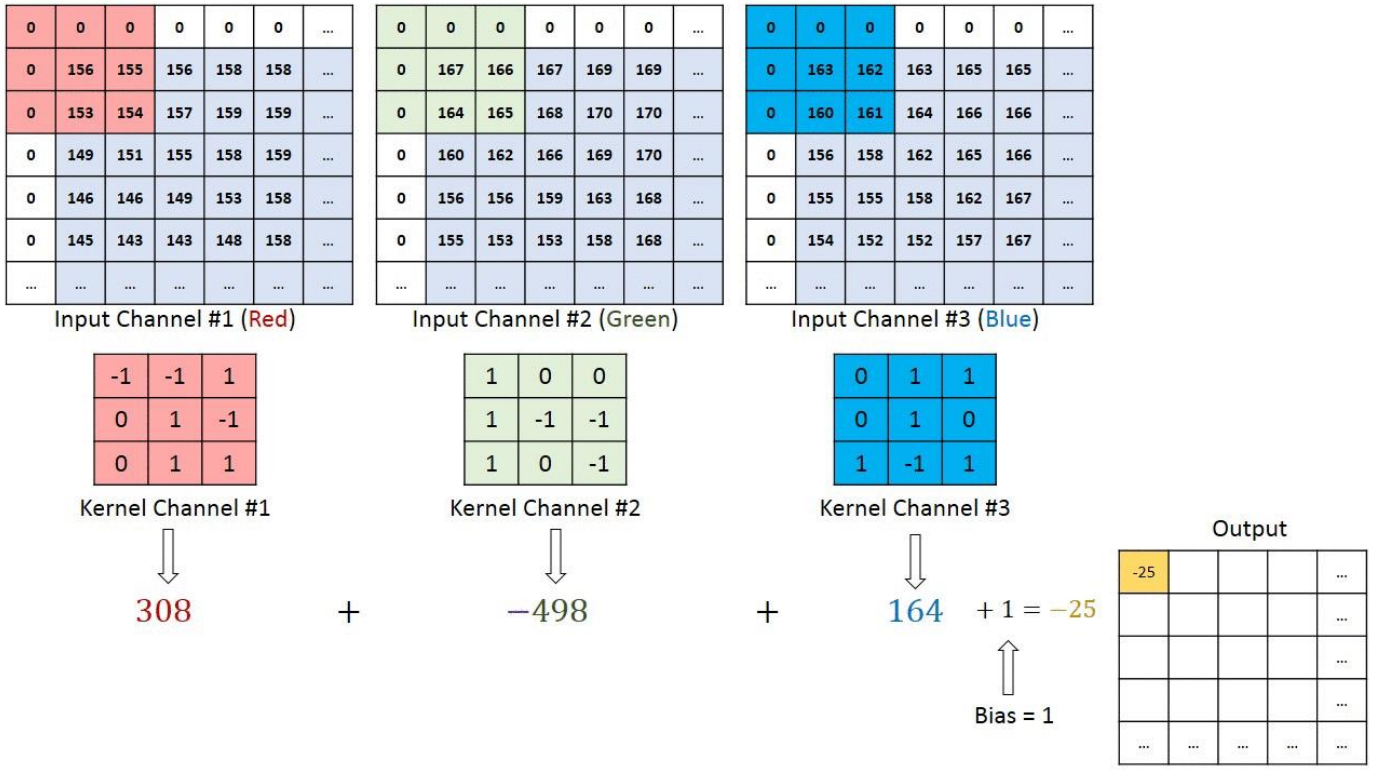
Padding de un píxel · Primera posición · Siguiente posición

$$N_{\text{salida}} = \lfloor (N + 2P - K)/S \rfloor + 1$$

Stride padding y tamaño de salida

Canales filtros y parámetros

- En una entrada RGB, una misma ventana contiene tres matrices. Un filtro estándar tiene pesos para las tres. Se calcula un producto y suma en cada canal, se suman los aportes de los canales y se añade un sesgo. Ese total produce una sola respuesta espacial del filtro.
- Con F filtros se obtienen F mapas. La cantidad de parámetros de una convolución estándar con sesgo es $N_{\text{parámetros}} = (k_h k_w C_{\text{in}} + 1)F$.

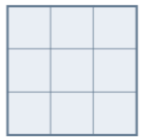


Parámetros de una capa convolucional

Convolución estándar con un sesgo por filtro

A Entrada en gris

8 filtros de 3 × 3



Un filtro de 3 × 3 × 1

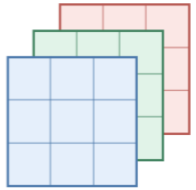
$$(3 \times 3 \times 1 + 1) \times 8$$

9 pesos + 1 sesgo por filtro

80 parámetros

B Entrada RGB

8 filtros de 3 × 3



Un filtro de 3 × 3 × 3

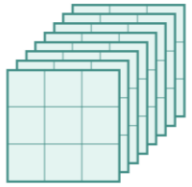
$$(3 \times 3 \times 3 + 1) \times 8$$

27 pesos + 1 sesgo por filtro

224 parámetros

C Entrada de 8 canales

16 filtros de 3 × 3



Un filtro de 3 × 3 × 8

$$(3 \times 3 \times 8 + 1) \times 16$$

72 pesos + 1 sesgo por filtro

1 168 parámetros

D Entrada RGB

16 filtros de 1 × 1



Un filtro de 1 × 1 × 3

$$(1 \times 1 \times 3 + 1) \times 16$$

3 pesos + 1 sesgo por filtro

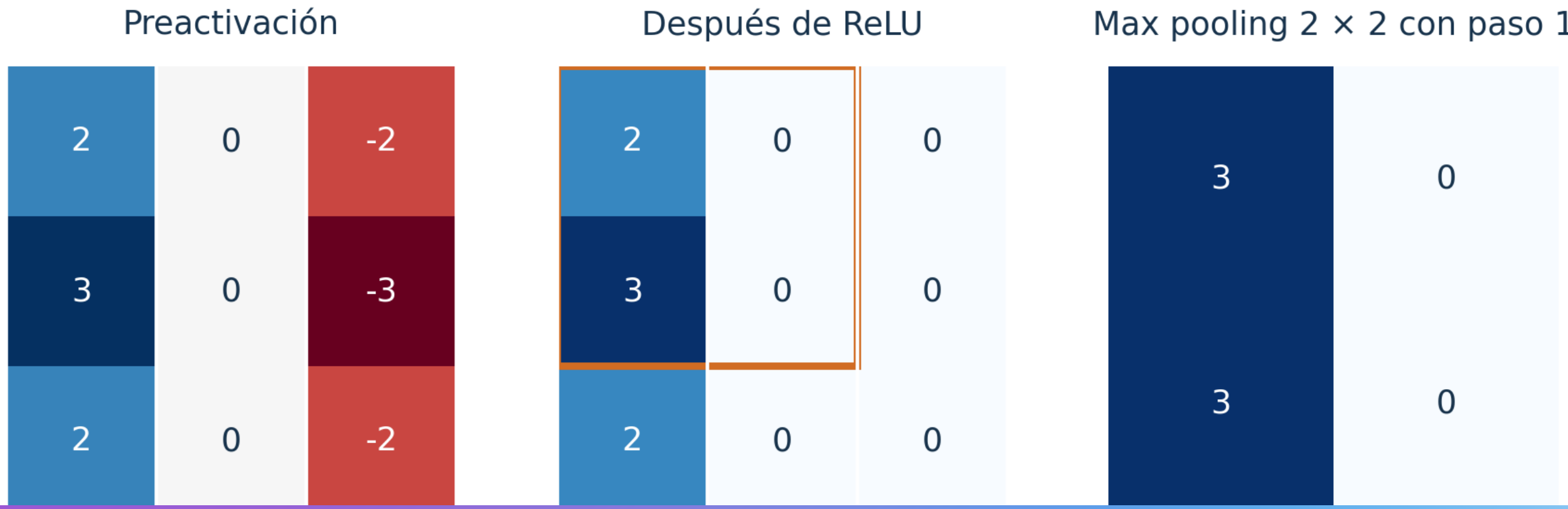
64 parámetros

Parámetros = (alto del kernel × ancho del kernel × canales de entrada + 1) × número de filtros

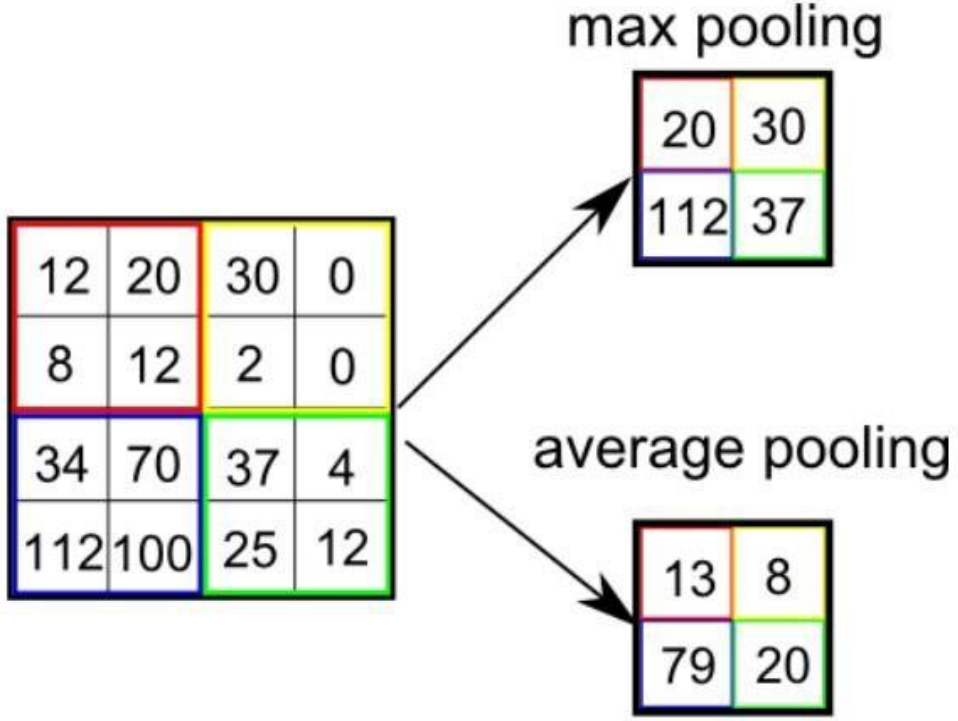
Canales filtros y parámetros

Capa de Pooling

- La capa de **pooling** reduce el tamaño espacial del mapa de características obtenido mediante la convolución. Al reducir su alto y ancho, disminuye el costo computacional de procesar los datos en las capas siguientes.
- Además, ayuda a conservar las características más destacadas y puede reducir la sensibilidad del modelo a pequeños desplazamientos, lo que puede facilitar el entrenamiento.



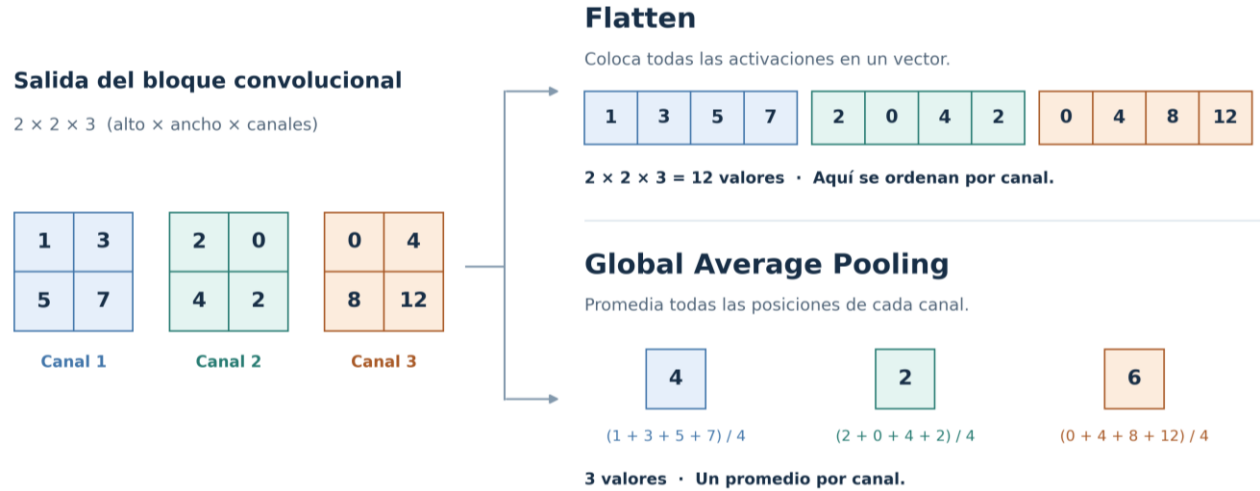
Capa de Pooling



- El pooling transforma cada canal por separado. **Max pooling** devuelve el máximo de una ventana. **Average pooling** devuelve su promedio.
- En sus versiones habituales tienen una regla fija y carecen de parámetros entrenables. La ventana y el stride son hiperparámetros.

Flatten y global average pooling

- Después de varias capas, tenemos un tensor $H' \times W' \times C'$ (comúnmente llamado deep features).
- **Flatten** reorganiza sus valores en un vector de longitud $H'W'C'$. Ese vector puede alimentar una o varias capas densas para la clasificación/regresión final.
- **Global average pooling**, promedia todas las posiciones de cada canal y entrega un valor por canal.

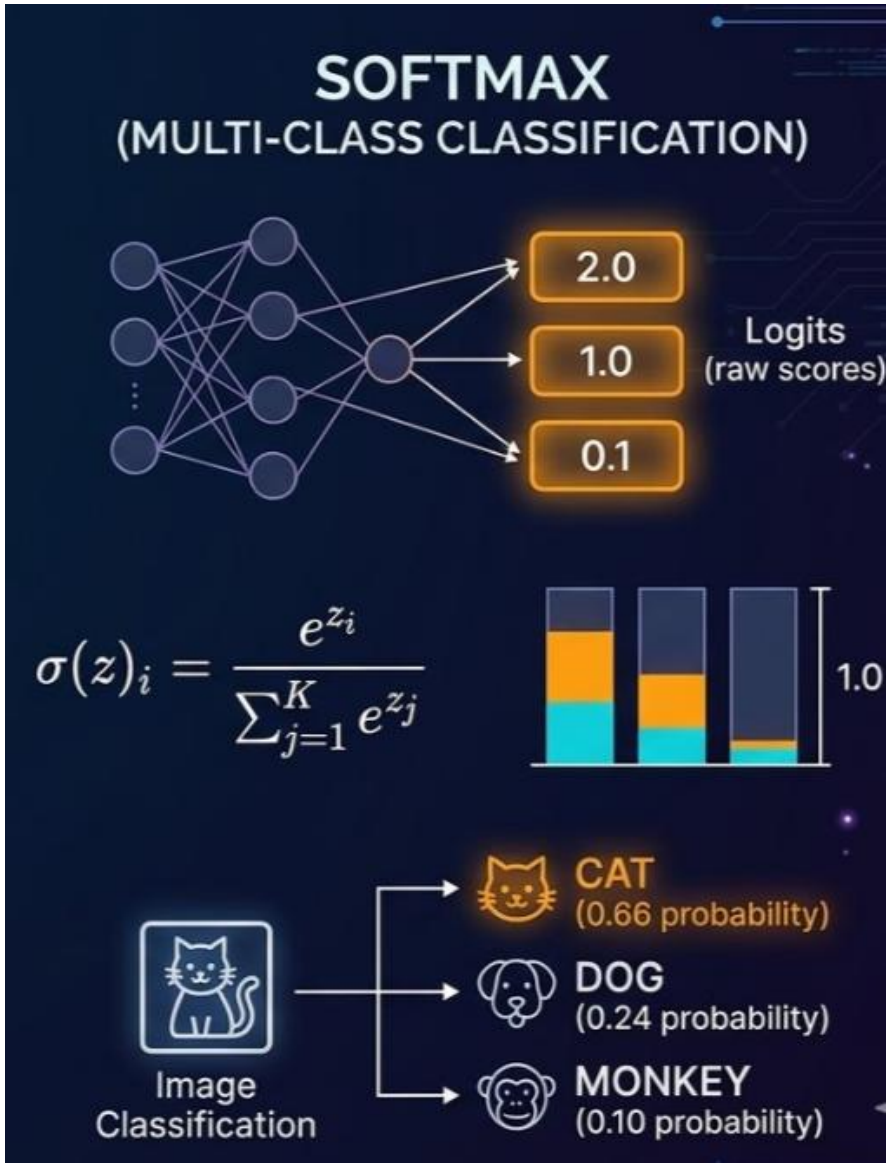


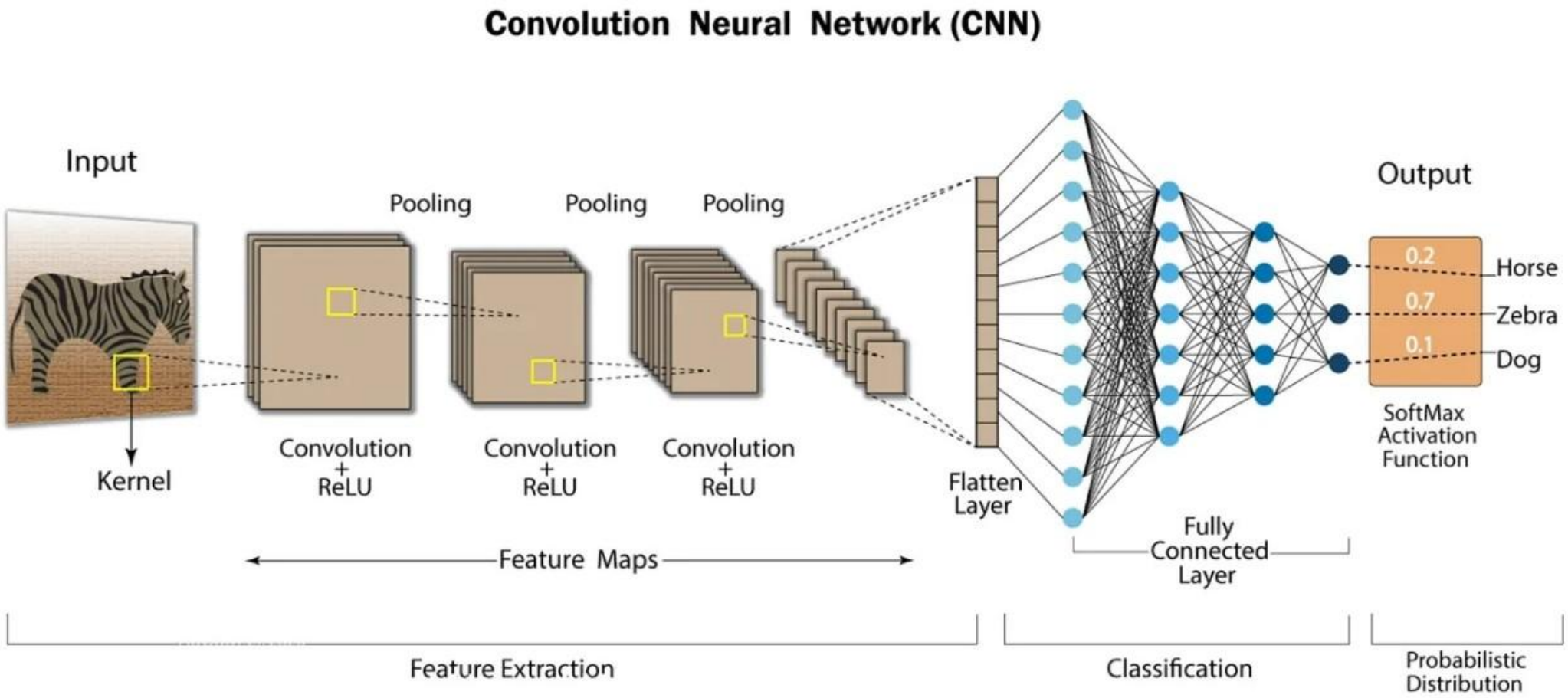
Clasificación final con capas densas

- El extractor convolucional transforma la imagen en características. Una cabeza de clasificación las convierte en logits. Esa cabeza puede ser una MLP pequeña.
- En clasificación cada componente del vector final corresponde a una clase.
- Para K clases mutuamente excluyentes, softmax transforma los logits en probabilidades que suman uno.

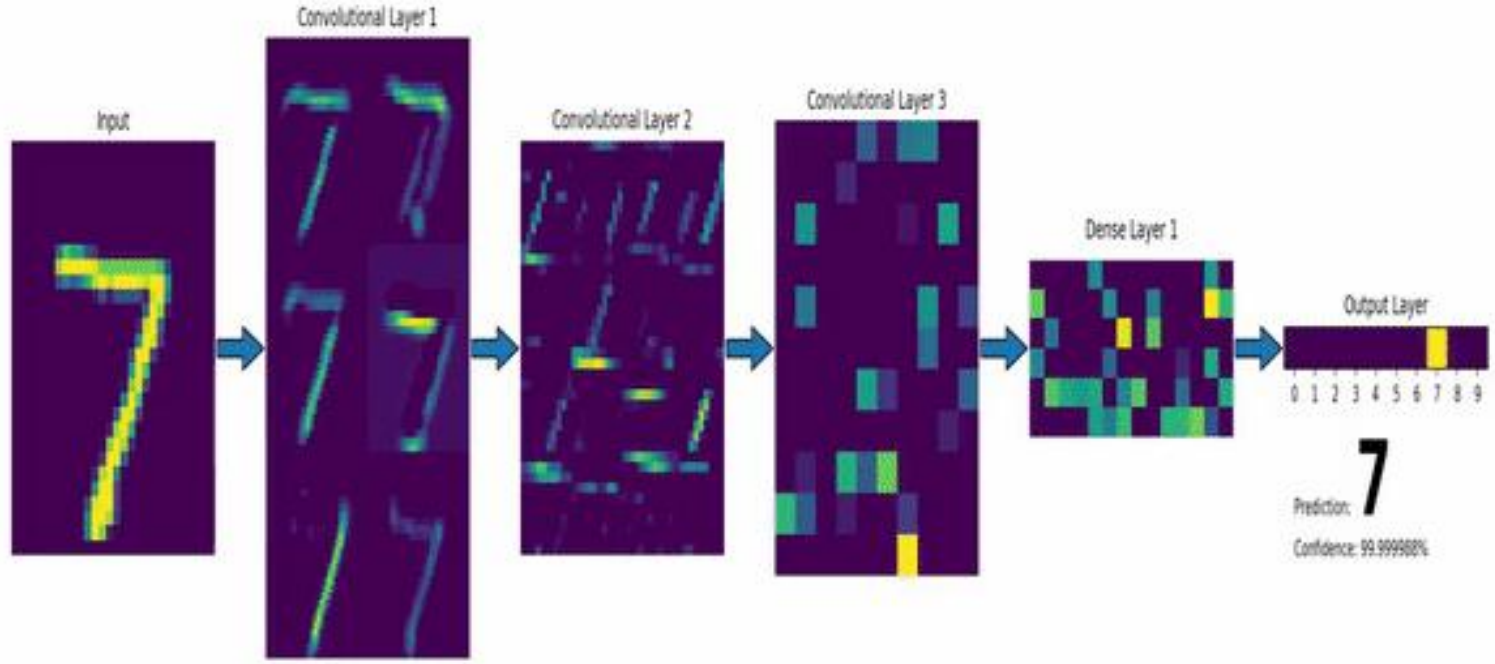
$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}.$$

- La clase predicha es la de mayor probabilidad.





Arquitectura de una CNN



Ejemplo en digits MNIST

Regularización y uso de redes preentrenadas

Aumento de datos

- Aplica el aumento después de separar las particiones y únicamente a entrenamiento durante el ajuste. Las imágenes de validación y prueba siguen un procedimiento fijo que permita comparar modelos.

Dropout y normalización

- **Dropout** desactiva aleatoriamente activaciones durante entrenamiento según una tasa configurada. Su objetivo es regularizar el aprendizaje.
- **Batch normalization** normaliza activaciones con estadísticas y una transformación de escala y desplazamiento.

Transfer learning

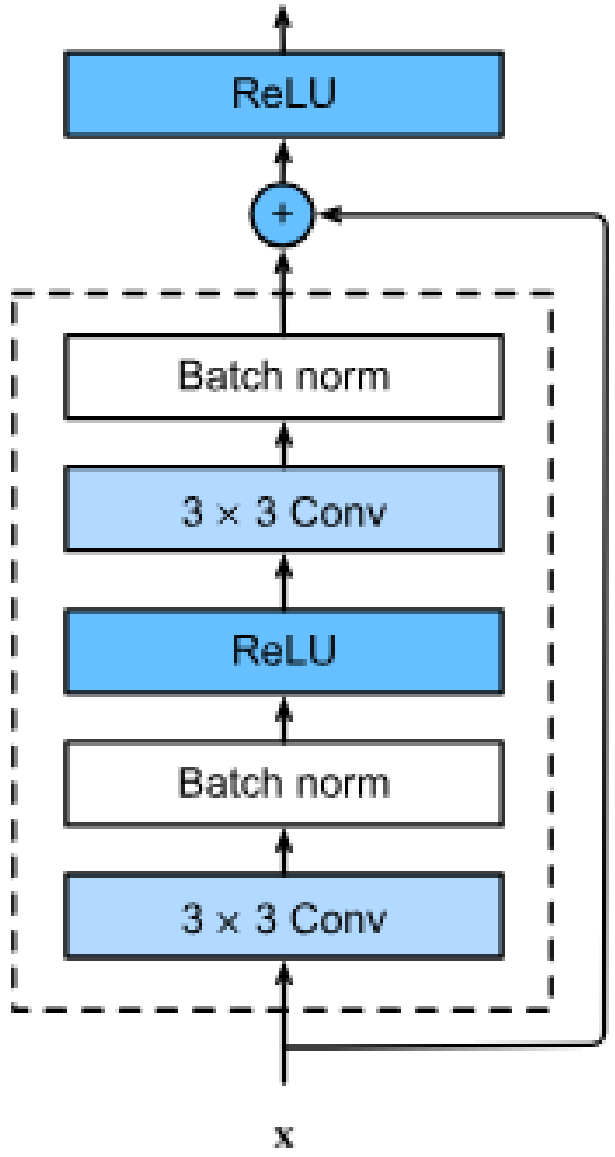
- Con pocos datos etiquetados, puede resultar útil empezar con un extractor preentrenado. Una primera fase conserva ese extractor y ajusta una nueva cabeza para las clases de interés. Otra fase puede afinar parte de sus pesos con una tasa menor.

AlexNet

- En 2012, la competencia de ImageNet consistía en clasificar fotografías en **1 000 categorías**. Un perro, por ejemplo, podía ocupar casi toda la foto o aparecer en una esquina, visto desde distintos ángulos y con fondos diferentes. El modelo tenía que reconocerlo en todos esos casos.
- Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton entrenaron una **red convolucional profunda (5 capas y 60M) en dos GPU**. Para la competencia, combinaron las predicciones de varias redes.
- El equipo ganó con **15.3 % de error top-5**, frente al **26.2 %** del segundo lugar. El salto mostró el potencial de las CNN profundas cuando se entrenan con grandes conjuntos de imágenes y suficiente capacidad de cómputo.

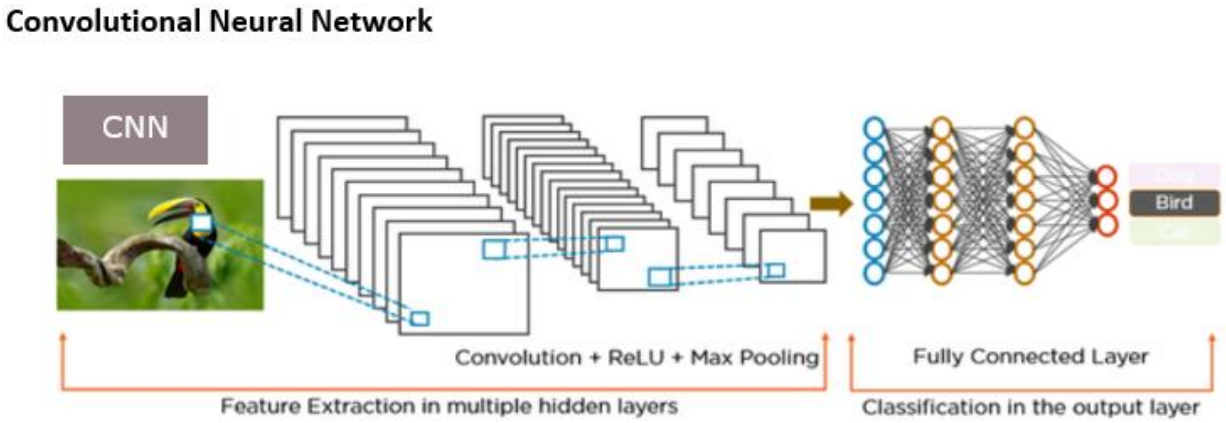
ResNet

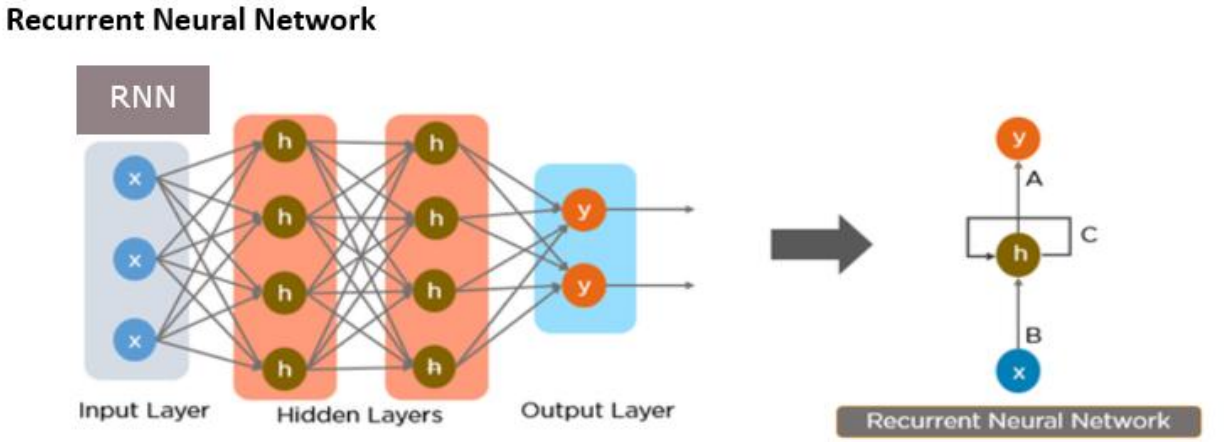
- Después de AlexNet, una pregunta natural era **¿qué pasa si seguimos agregando capas?**
- Al hacerlo, Kaiming He y sus colaboradores encontraron algo sorprendente. En algunas redes, agregar capas hacía que el modelo ajustara peor.
- Su propuesta fue darle a cada bloque un camino directo por el que pudiera pasar la entrada. Las capas del bloque aprendían qué corrección sumar a esa entrada. Si hacía falta conservarla, podían aprender una corrección cercana a cero. Con esa idea, el equipo entrenó redes de hasta **152 capas (60M)** y ganó ImageNet 2015 con un ensamble de redes residuales.

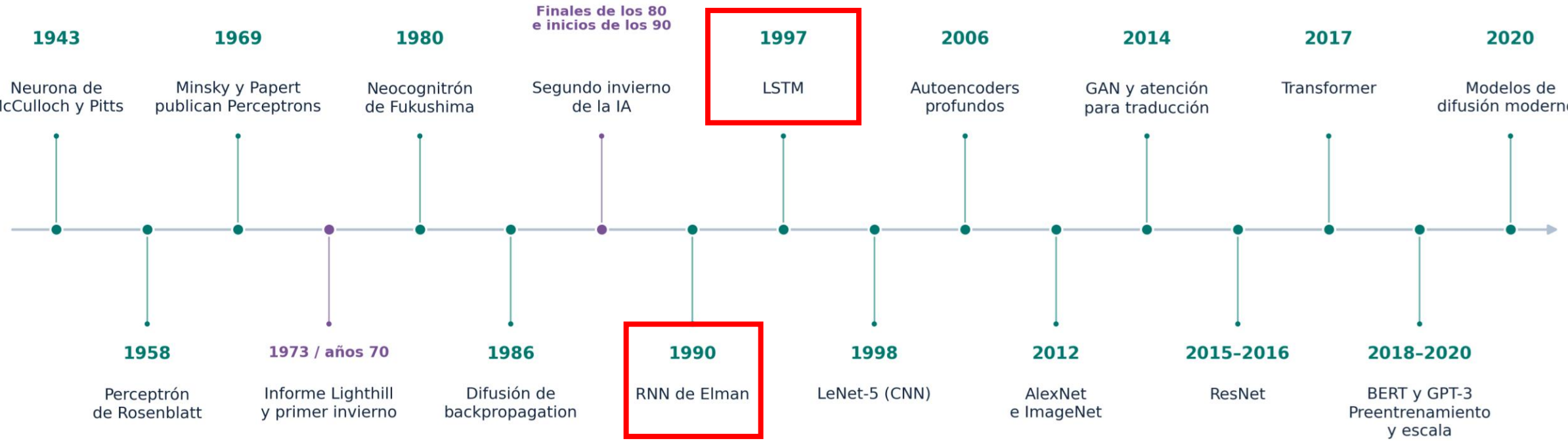


De las CNN a las redes recurrentes (RNN)

- Con las CNN aprendimos a buscar patrones locales.
- **En una secuencia, el orden también importa.** Una misma medición puede indicar que una señal está subiendo o bajando, según lo que ocurrió en los pasos anteriores.






1943
Neurona de McCulloch y Pitts
1958
Perceptrón de Rosenblatt
1969
Minsky y Papert publican Perceptrons
1973 / años 70
Informe Lighthill y primer invierno
1980
Neocognitrón de Fukushima
1986
Difusión de backpropagation
Finales de los 80 e inicios de los 90
Segundo invierno de la IA
1990
RNN de Elman
1997
LSTM
1998
LeNet-5 (CNN)
2006
Autoencoders profundos
2012
AlexNet e ImageNet
2014
GAN y atención para traducción
2015–2016
ResNet
2017
Transformer
2018–2020
BERT y GPT-3 Preentrenamiento y escala
2020
Modelos de difusión modernos
Hito técnico
Contexto histórico de la IA
Orden cronológico con espaciado uniforme